

**RĪGAS TEHNISKĀ UNIVERSITĀTE  
DATORZINĀTNES, INFORMĀCIJAS TEHNOLOĢIJAS UN  
ENERĢĒTIKAS FAKULTĀTE  
VIEDĀS ELEKTRONISKĀS SISTĒMAS**

**Kursa darbs studiju kursā  
Elektronisko vadības sistēmu projektēšana**

**ČETRU KĀJU ROBOTS**

Bakalaura studiju programmas

“Viedās elektroniskās sistēmas”

1. kursa RECVO 2. grupas students,

Riks Sipovičs

Studenta apliecības Nr. 241REC061

**RĪGA 2025**

# SATURS

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>IEVADS .....</b>                                                | <b>3</b>  |
| <b>1. Literatūras apskats .....</b>                                | <b>4</b>  |
| 1.1. Daudzkāju robotu klasifikācija un pielietojuma jomas.....     | 4         |
| <b>2. Robota konstrukcijas un sistēmas raksturojums.....</b>       | <b>6</b>  |
| 2.1. Robota konstrukcijas apraksts: rāmis, kājas, materiāli .....  | 6         |
| 2.2. Izmantotie elektronikas komponenti.....                       | 8         |
| 2.3. Elektroniskās pieslēguma plates izveide.....                  | 10        |
| 2.4. Enerģijas padeves un barošanas risinājumi .....               | 12        |
| <b>3. Vadības sistēmas projektēšana un programmēšana.....</b>      | <b>13</b> |
| 3.1. Izmantotas bibliotēkas .....                                  | 13        |
| 3.2. OLED displejs.....                                            | 14        |
| 3.3. Servo motori un to vadība.....                                | 15        |
| 3.4. Komandu sistēma.....                                          | 16        |
| 3.5. Robota pielietojums, efektivitāte un uzlabojumu iespējas..... | 19        |
| <b>SECINĀJUMI .....</b>                                            | <b>20</b> |
| <b>PIELIKUMI .....</b>                                             | <b>21</b> |
| <b>IZMANTOTA LITERATŪRA.....</b>                                   | <b>36</b> |

## IEVADS

**Darba autors:** Riks Sipovičs

**Pētījuma objekts:** Četru kāju robota konstrukcija un elektroniskā vadības sistēma

**Kursa darba mērķis:** izstrādāt funkcionējošu četru kāju robotu, izveidojot tā mehānisko konstrukciju un izstrādājot elektroniskās vadības sistēmu ar mikrokontrolieri.

### Kursa darba uzdevumi:

- izpētīt līdzīgus robotikas risinājumus un to pielietojumu dažādās nozarēs;
- projektēt četru kāju robota mehānisko struktūru, izvēloties atbilstošus materiālus un komponentes;
- izstrādāt elektronisko vadības sistēmu, kas nodrošina stabilu un koordinētu kustību;
- testēt un analizēt robota darbību, veicot nepieciešamos pielāgojumus.

**Darba metodes:** literatūras un tehniskās dokumentācijas analīze, 3D modelēšana ar Tinkercad, 3D drukāšana elektronisko shēmu izstrāde, programmēšana un praktiskie testi.

Mūsdienu tehnoloģiju attīstība ievērojami paplašina iespējas robotikas jomā, kur viena no interesantākajām un praktiski pielietojamām nozarēm ir daudzkāju roboti. Tie tiek izmantoti sarežģītā reljefā, meklēšanas un glābšanas operācijās, kā arī izglītības un pētniecības mērķiem. Četru kāju roboti izceļas ar stabilitāti, kustību elastību un salīdzinoši vienkāršu konstrukciju, padarot tos par labu izvēli individuāliem projektiem un pirmajiem soļiem autonomās mobilitātes jomā.

Robota darbību nodrošina programmēta vadības sistēma, kas balstīta uz Arduino Nano mikrokontrolieri. Kods izstrādāts Arduino IDE vidē, izmantojot vairākas ārējās bibliotēkas, kas nodrošina ērtu saskarni ar servo motoriem, OLED displeju un seriālo komandu apstrādi. Programmatūra ir strukturēta vairākos loģiskos blokos, tostarp inicializācija, servo motoru apkalpošana, displeja animācijas un komandu izpilde. Vadības algoritmi ir veidoti tā, lai nodrošinātu sinhronizētu četru kāju kustību, kas ir būtiska stabilai gaitai.

## 1. Literatūras apskats

### 1.1. Daudzkāju robotu klasifikācija un pielietojuma jomas

Daudzkāju roboti tiek klasificēti pēc to kāju skaita, konstrukcijas veida un pielietojuma jomas. Četru kāju roboti, kā šajā darbā aprakstītais, ir izplatīti mācību, pētniecības un hobija projektos. Tie nodrošina stabilu kustību dažādos apstākļos un ir salīdzinoši vienkārši uzbūvē.



1.1. attēls Četru kāju robot Xiaomi

Šādi roboti tiek izmantoti, lai testētu gaitas algoritmus, apgūtu mehatroniku, kā arī attīstītu tālvadības un autonomās vadības sistēmas. Praksē tie var kalpot arī kā platforma sarežģītākām funkcijām, piemēram, vides monitorēšanai vai vienkāršām glābšanas operācijām nelīdzenā reljefā. Robota elastīgā pielāgojamība padara to par noderīgu rīku gan izglītībā, gan praktiskos pielietojumos.



1.2. attēls ETH Zurich “ANYmal”

Četru kāju robotu un daudzkāju kustības sistēmu izpēte pēdējo gadu laikā ir kļuvusi par nozīmīgu pētniecības virzienu robotikas nozarē. Literatūrā plaši aplūkotas

daudzkāju gaitas priekšrocības, piemēram, stabilitāte, spēja pielāgoties dažādiem reljefiem un nepārtrauktas saskares iespēja ar virsmu, kas atšķir šos robotus no riteņtransporta sistēmām. Tādi roboti kā Boston Dynamics “Spot” vai ETH Zurich “ANYmal” pierāda, ka četru un sešu kāju konstrukcijas ir piemērotas gan industriālai, gan pētnieciskai videi.



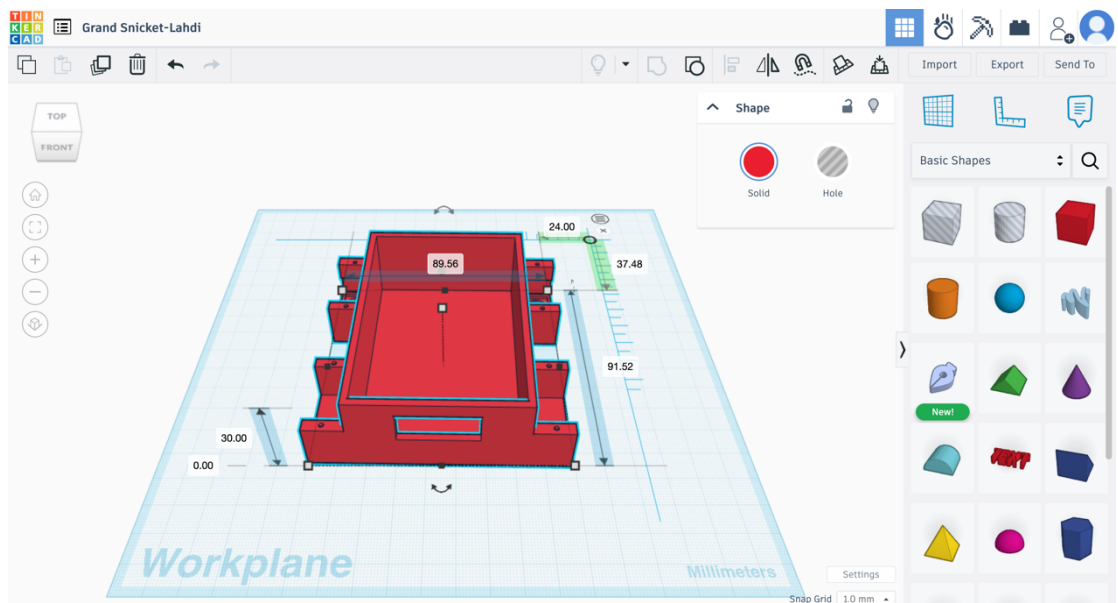
1.3. attēls Robots Boston Dynamics “Spot”

Daudzi akadēmiski darbi apraksta četru kāju robota gaitas algoritmus, kuru pamatā ir apgrieztā kinemātika un soļu ciklu sinhronizācija. Tāpat bieži izmantoti Arduino mikrokontrolieri kā platforma vienkāršiem prototipiem, pateicoties to pieejamībai, plašajai bibliotēku bāzei un izstrādātāju kopienai.

## 2. Robota konstrukcijas un sistēmas raksturojums

### 2.1. Robota konstrukcijas apraksts: rāmis, kājas, materiāli

Robota rāmis un kājas ir izgatavotas ar 3D printeri no PLA plastmasas. Visas konstrukcijas detaļas izveidotas ar Tinkercad palīdzību vai atrastas tiešsaistē, pielāgotas konkrētajam dizainam.

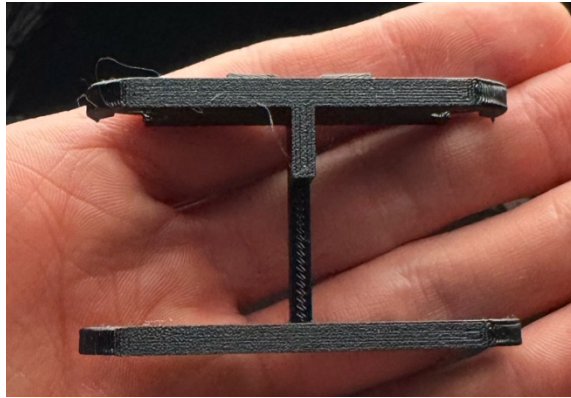


2.1.attēls Tinkercad izveidots robota korpuss

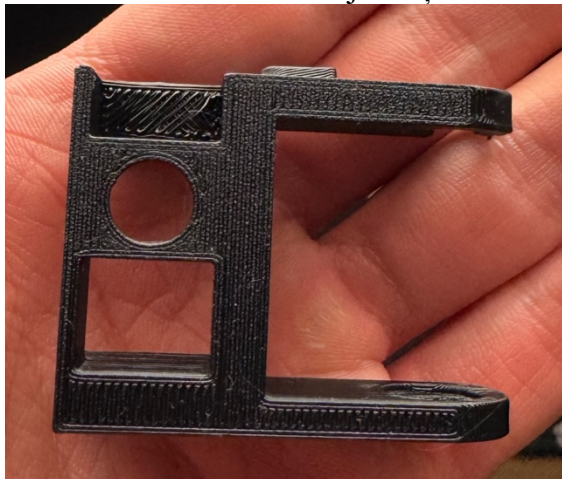
Četras kājas veidotas ar trīs locītavām katrā, nodrošinot nepieciešamās kustības brīvības pakāpes.



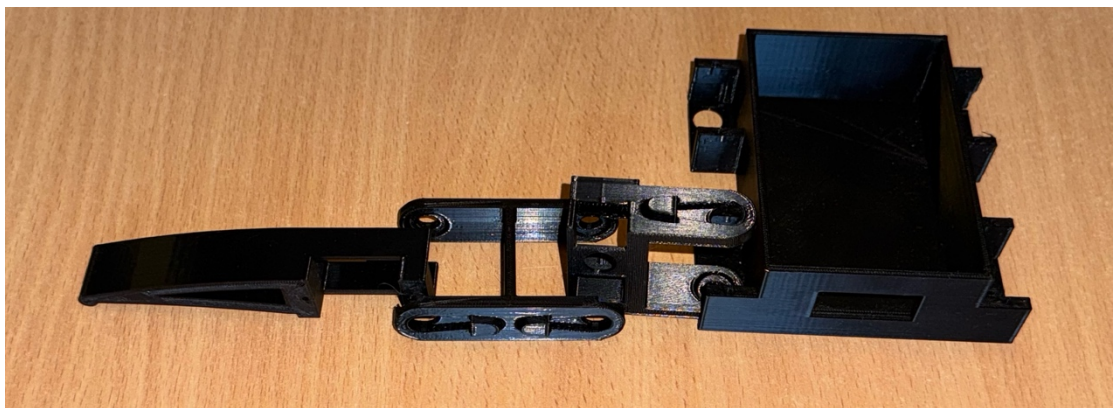
2.2.attēls Pirmā kājas daļa



2.3.attēls Otrā kājas daļa



2.4.attēls Trešā kājas daļa



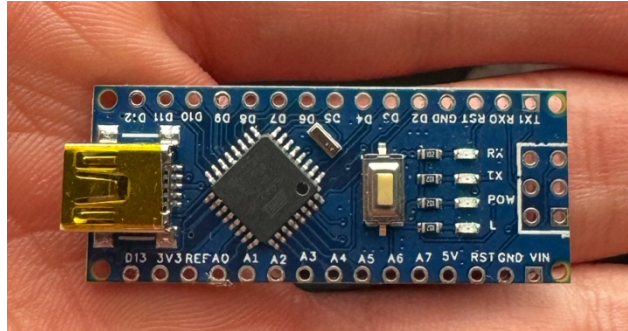
2.5. attēls Viena robota kāja

Rāmis ir kompakts un viegls, nodrošinot mobilitāti un svara balansu. Kājas piestiprinātas tā, lai nodrošinātu vienmērīgu slodzes sadalījumu un labu saķeri ar virsmu. Šāds konstrukcijas risinājums padara robotu piemērotu gan iekštelpu, gan vienkāršiem āra testiem, vienlaikus atvieglojot detaļu nomaiņu vai uzlabošanu.



## 2.2. Izmantotie elektronikas komponenti

Četru kāju robota darbību nodrošina vairākas elektroniskās sastāvdaļas, kas apvienotas vienotā vadības sistēmā. Sistēmas centrā atrodas **Arduino Nano** mikrokontrolieris, kurš vada **12 SG90 servomotorus**, nodrošinot katras kājas kustību trīs brīvības pakāpēs.

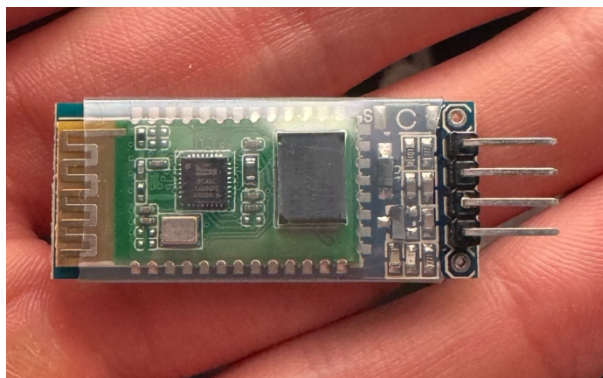


2.6.attēls Arduino Nano



2.7.attēls SG90 servomotors

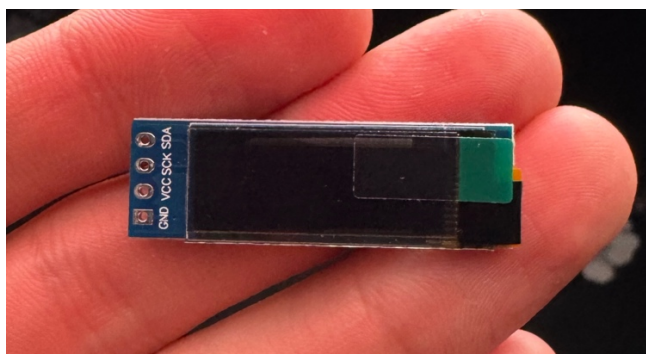
Datu apmaiņai un attālai vadībai tiek izmantots **CH-06 Bluetooth modulis**, kas ļauj robotam pieņemt komandas no datora vai mobilās ierīces bezvadu režīmā.



2.8.attēls CH-06 Bluetooth modulis

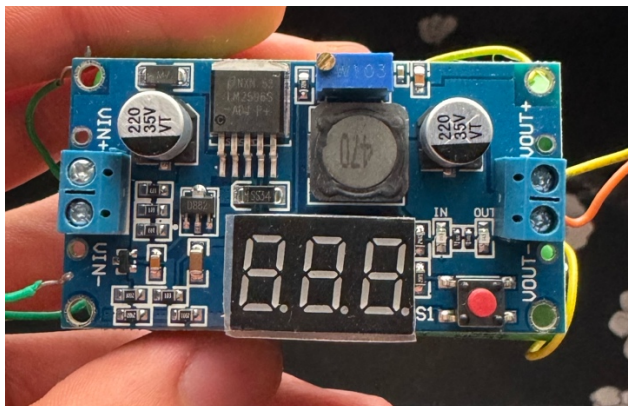


Vizuālais interfeiss sastāv no **OLED I2C displeja (128x32)**, uz kura tiek attēlotas animētas “acis”.

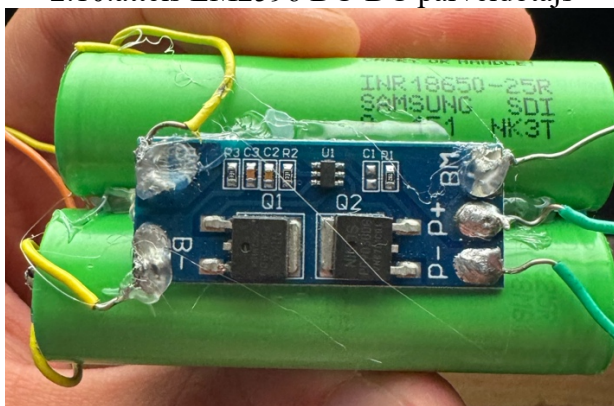


2.9.attēls OLED I2C displejs

Barošanas sistēma balstīta uz **diviem 10659 Li-ion 20 A akumulatoriem**, kas savienoti caur **BMS 2S plati**. Sprieguma stabilizēšanai tiek izmantots **LM2596 DC-DC pārveidotājs**.



2.10.attēls LM2596 DC-DC pārveidotājs

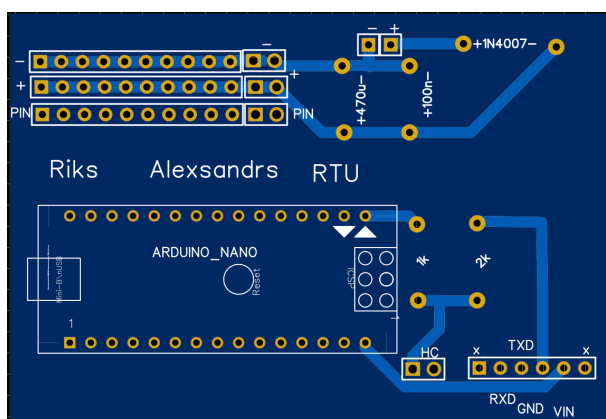


2.11.attēls 10659 Li-ion 20 A akumulatori ar bms plati

Papildus ir iekļauti **kondensatori**, lai nodrošinātu stabilu spriegumu un izvairītos no traucējumiem pie pēkšņām slodzēm.

### 2.3. Elektroniskās pieslēguma plates izveide

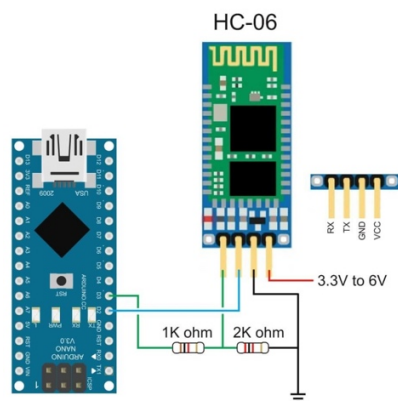
Lai nodrošinātu ērtu un uzticamu visu elektronisko komponentu pieslēgšanu četru kāju robotam, tika izstrādāta divpusēja elektroniskā plate, kurā integrēti visi svarīgākie savienojumi. Plates mērķis ir atvieglot 12 SG90 servomotoru, Arduino Nano, barošanas avota un Bluetooth CH-06 moduļa savienošanu vienuviet. Tā ļauj ievērojami samazināt vadu jucekli, kā arī uzlabo sistēmas stabilitāti un servisu iespējas testēšanas un pielāgošanas laikā (Skat. 2.12.attēls).



2.12.attēls EasyEDA izveidota plate

Plates dizains tika izveidots, izmantojot EasyEDA – lietotājam draudzīgu tiešsaistes elektronikas projektēšanas rīku (skat. 1.pielikums). Tajā tika izveidots izkārtojums ar skaidri marķētiem izvadiem, tostarp servo motoru pieslēgšanas pin-header rindu, atsevišķiem spraudņiem Bluetooth moduļa pieslēgšanai (TXD, RXD, VIN, GND), kā arī rezistoru un kondensatoru pozīcijām. Uz plates paredzēta vieta arī 1N4007 aizsardzības diodei un 470  $\mu$ F un 100 nF kondensatoriem, lai stabilizētu spriegumu.

Bluetooth modulim HC-06 nepieciešams pieslēgt TX signālu no Arduino uz HC-06 RX ieeju, taču tā kā Arduino darbojas ar 5V loģiku un HC-06 RX pieslēgvietā paredzēta 3.3V. Divi rezistori – 2 k $\Omega$  un 1 k $\Omega$  – veido sprieguma dalītāju, kas samazina signālu no 5V uz ~3.3V (Skat. 2.13.attēls). Tas aizsargā Bluetooth moduli no pārslodzes un bojājumiem.



2.13.attēls Bluetooth moduļa HC-06 pieslēgumā shēma

Lai iegūtu fizisku plati, tā tika pasūtīta no JLCPCB – Ķīnas elektronikas ražotāja, kas piedāvā augstas precizitātes, ātru un zemu izmaksu PCB izgatavošanu pēc Gerber failiem.



2.14.attēls JLCPCB logotips

Pasūtītā plate tika piegādāta kvalitatīvā formā, ar precīzu slāņu izvietojumu un caurumu izvietojumu saskaņā ar EasyEDA specifikāciju. Plates izmantošana būtiski atvieglo robota montāžas procesu, padarot pieslēgumu uzticamu, kompakti organizētu un atkārtoti savienojamu. Tā kalpo kā platforma arī turpmākai sistēmas attīstībai – iespējams pievienot sensorus vai citus moduļus bez nepieciešamības pārveidot vadu konfigurāciju.

## 2.4. Energijas padeves un barošanas risinājumi

Četru kāju robota enerģijas padeve ir izveidota, lai nodrošinātu stabilu un drošu darbību gan mikrokontrolierim, gan servomotoriem. Robota barošanai tiek izmantoti **divi 10659 litija jonu akumulatori 20A**, kas savienoti **2S konfigurācijā**, iegūstot nominālo spriegumu aptuveni 7,4V. Akumulatori ir pieslēgti caur **BMS (Battery Management System) 2S plati**, kas nodrošina aizsardzību pret pārlādi, pārdziļinātu izlādi, strāvas pārsniegšanu un sabalansē katra elementa uzlādi.

Tā kā servomotori un Arduino darbojas ar 5V spriegumu, sistēmā integrēts **sprieguma pārveidotājs LM2596**, kas pazemina akumulatoru spriegumu uz nepieciešamajiem 5V. LM2596 modulis spēj nodrošināt pietiekamu jaudu vienlaikus visiem servomotoriem. Lai pasargātu sistēmu no strāvas svārstībām pie pēkšņas servo slodzes, tiek izmantoti **elektrolītiskie kondensatori**, kas stabilizē barošanas ķēdi un uzlabo kopējo sistēmas uzticamību.

Elektroniskajā platē tika izmantota **1N4007** taisngrieža **diode**, lai aizsargātu barošanas ķēdi no polaritātes kļūdām. Tā ļauj strāvai plūst tikai vienā virzienā, novēršot bojājumus, ja akumulators pievienots nepareizi. Šī diode darbojas kā aizsardzības elements, nodrošinot sistēmas drošību.

### 3. Vadības sistēmas projektēšana un programmēšana

#### 3.1. Izmantotas bibliotēkas

Lai atvieglotu sarežģītu komponentu pārvaldību, programmā izmantotas šādas bibliotēkas:

**Servo.h** – nodrošina vienkāršu veidu servo motoru vadībai, nosūtot PWM signālus ar komandu **servo.write(*leņķis*)**; Tiek izmantota, lai kontrolētu katru no 12 servomotoriem.

**FlexiTimer2.h** – ļauj izveidot periodisku apkalpošanas funkciju, kas izsaucas ik pēc 20 ms un pārvieto servus pa maziem soļiem uz mērķa pozīciju. Tas nodrošina vienmērīgu, lineāru kāju kustību.

**SerialCommand.h** – pārvalda un analizē seriālās komandas, kuras tiek ievadītas no datora vai Bluetooth ierīces.

**Adafruit\_GFX.h** un **Adafruit\_SSD1306.h** – tiek izmantotas OLED displeja vadībai. Šīs bibliotēkas ļauj zīmēt grafiskas formas, tekstus un animācijas ar minimālu kodu.

### 3.2. OLED displejs

OLED displejs un tā darbība Robots aprīkots ar 128x32 pikseļu I2C OLED displeju, kas attēlo divas animētas acis. Displejs pievienots, izmantojot I2C protokolu, un darbojas ar Adafruit SSD1306 un GFX bibliotēkām. Zīlīšu pozīcijas tiek dinamiskai mainītas, simulējot skatīšanos uz augšu, leju, pa labi vai pa kreisi. Displeja funkcionalitāte ir definēta vairākās funkcijās (**drawEyes()**, **center()**, **blink()**, u.c.), kas tiek izsauktas cikliskā veidā galvenajā **loop()** funkcijā. Šī vizuālā komponenta mērķis ir padarīt robotu lietotājam draudzīgāku, kā arī sniegt nelielu atgriezenisko saiti par sistēmas darbību.

Koda piemērs:

```
display.fillCircle(EXR, vpos, EYE, WHITE); // labā acs
display.fillCircle(EXL, vpos, EYE, WHITE); // kreisā acs
display.fillCircle(EXR+plh, vpos+plv, PUPIL, BLACK); // labās zīlītes pozīcija
display.fillCircle(EXL+prh, vpos+plv, PUPIL, BLACK); // kreisās zīlītes pozīcija
display.display();
```

Displeja animācijas tiek realizētas, izmantojot dažādu **fillCircle()**, **drawCircle()** un **fillRect()** funkciju kombināciju, kuras nodrošina bibliotēka **Adafruit\_GFX**. Attēlojot zīlītes dažādās pozīcijās, tiek imitēta skatiena kustība: uz augšu **up()**, uz leju **down()**, uz sāniem **left()**, **right()** un neizpratne **confuse()**, kur zīlītes novietotas asimetriski. Lai panāktu animācijas plūdumu, katra kustība tiek papildināta ar **display.display();** komandu, kas atjauno redzamo ekrāna saturu. Tādā veidā displejs darbojas kā papildus interfeiss, sniedzot vizuālu informāciju par robota stāvokli bez nepieciešamības izmantot papildus LED indikatorus vai skaņas signālus. Tas ir īpaši noderīgi interaktīvā vai izglītojošā vidē, kur vizualitāte spēlē būtisku lomu. (Skat. 4.pielikums)

Koda piemērs:

```
void right(int, vpos){
    drawEyes(3, 3, 0, vpos); // Zīlītes nobīdītas uz labo pusi
    display.display();       // Atjauno displeja saturu
    delay(1000);             // Aizture efektam
}
```



### 3.3. Servo motori un to vadība

Katra robota kāja aprīkota ar trīs SG90 servomotoriem, kopumā 12 motori. Tie nodrošina trīs brīvības pakāpes – pleca kustību, elkoņa kustību un kājas rotāciju. Motori pieslēgti Arduino digitālajiem izvadēm un tiek vadīti, izmantojot PWM signālus. Servo kustību loģika ir balstīta uz koordinātu sistēmu. Katras kājas gala punkta (x, y, z) kustības mērķis tiek pārrēķināts par leņķiem (alpha, beta, gamma), kas nodrošina pareizu servomotoru pozīciju. Kustības notiek pakāpeniski, lai nodrošinātu vienmērīgu gaitas kustību bez saraustījumiem. Šo funkcionalitāti realizē taimeris ar FlexiTimer2.

Koda piemērs:

```
servo[leg][0].write(alpha); // plecs
servo[leg][1].write(beta);  // elkonis
servo[leg][2].write(gamma); // griešana
```

Katrs servomotors tiek inicializēts un pieslēgts attiecīgajam digitālajam izvadam ar šādu ciklu:

```
for (int i = 0; i < 4; i++) {
  for (int j = 0; j < 3; j++) { servo[i][j].attach(servo_pin[i][j]);
  delay(100);
}
}
```

Šeit **servo[4][3]** ir divdimensiju masīvs, kur i norāda kāju (0–3), un j – kustības asi (0 – gūža, 1 – plecs, 2 – ceļš). Pēc pieslēgšanas katru servo var vadīt, nosūtot tam vēlamu leņķi, piemēram: **servo[0][0].write(90);** – tas nozīmē, ka 1. kājas gūžas servomotors pagriežas uz 90 grādiem. Servo kustības tiek aprēķinātas no kartēziešu koordinātām (x, y, z), kur tiek noteikts kājas gala punkts, un pēc tam ar **cartesian\_to\_polar()** funkciju tas tiek pārvērsts servo leņķos. Tas ļauj robotam veikt precīzas un gludas kustības telpā.

### 3.4. Komandu sistēma

Robots pieņem komandas gan no USB seriālā porta, gan caur CH6 Bluetooth moduli. Vadība notiek ar **SerialCommand** bibliotēku, kas nolasīto ievadi pārvērš noteiktās darbībās. Katru komandu veido kods "w", kam seko divi parametri: darbības veids un soļu skaits vai izpildes parametrs. Galvenās komandas:

w 0 1 - Piecelties;

w 0 0 - Apsēsties;

w 1 1 - Solis uz priekšu;

w 2 1 - Solis atpakaļ;

w 3 1 - Solis pa kreisi;

w 4 1 - Solis pa labi;

w 5 1 - Pamāj ar "roku" (kāju).

Koda **w 0 1** komandas piemērs:

```
void stand(void) {  
    move_speed = stand_seat_speed;  
    for (int leg = 0; leg < 4; leg++) {  
        set_site(leg, KEEP, KEEP, z_default);  
    }  
    wait_all_reach();  
}
```

Kad lietotājs nosūta komandu **w 4 1**, robots saņem instrukciju izpildīt vienu pagriezienu soli pa labi. Komandas interpretācija notiek ar **SerialCommand** palīdzību, kur funkcija **action\_cmd()** identificē komandas kodu 4 un izsauc **turn\_right(1)** funkciju. Šajā funkcijā tiek izmantots apgrieztais kinemātiskais modelis, lai pārvietotu kājas telpā un radītu pagriezienu efektu. Pagriešanās tiek panākta, secīgi pārvietojot divas kājas uz jaunu pozīciju, kamēr pārējās kalpo kā atbalsta punkti. Šī cikliskā pārvietošana tiek atkārtota vairākos soļos ar sinhronizētu kustību.

Koda **w 4 1** komandas piemērs:

```
void turn_right(unsigned int step)
{
    move_speed = spot_turn_speed;
    while (step-- > 0)
    {
        if (site_now[2][1] == y_start)
        {
            // Kāja 2 paceļas uz augšu
            set_site(2, x_default + x_offset, y_start, z_up);
            wait_all_reach();

            // Divas kājas uz jaunām pagrieziena pozīcijām
            set_site(0, turn_x0 - x_offset, turn_y0, z_default);
            set_site(1, turn_x1 - x_offset, turn_y1, z_default);
            set_site(2, turn_x0 + x_offset, turn_y0, z_up);
            set_site(3, turn_x1 + x_offset, turn_y1, z_default);
            wait_all_reach();

            // Kāja 2 nolaižas
            set_site(2, turn_x0 + x_offset, turn_y0, z_default);
            wait_all_reach();

            // Visas kājas atgriežas sākuma pozīcijās
            set_site(0, x_default + x_offset, y_start, z_up);
        }
    }
}
```

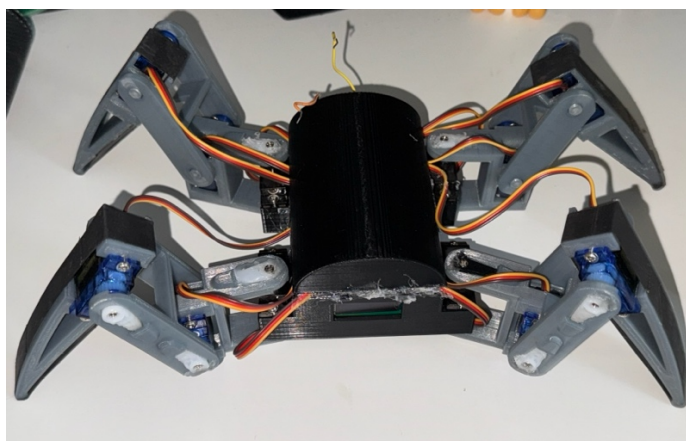
Svarīga daļa robota servo kustību kontroles loģikā ir funkcija **wait\_all\_reach()**, kas nodrošina, ka visas kājas pabeidz savu kustību pirms nākamā darbība tiek uzsākta. Tā ir bloķējoša funkcija, kas nozīmē, ka programma uz brīdi apstājas un gaida, līdz visas kājas sasniedz iepriekš noteiktās koordinātas. Šī funkcija izmanto iekšējo **wait\_reach(int leg)** apakšfunkciju, kas pārbauda, vai viena konkrēta kāja ir sasniegusi sev paredzēto gala punktu (**site\_expect**).

Koda **wait\_all\_reach** funkcijas piemērs:

```
void wait_reach(int leg) {  
    while (1)  
        if (site_now[leg][0] == site_expect[leg][0])  
            if (site_now[leg][1] == site_expect[leg][1])  
                if (site_now[leg][2] == site_expect[leg][2])  
                    break;  
}  
  
void wait_all_reach(void)  
{  
    for (int i = 0; i < 4; i++)  
        wait_reach(i);  
}
```

### 3.5. Robota pielietojums, efektivitāte un uzlabojumu iespējas

Četru kāju robots kalpo kā lielisks instruments robotikas, elektronikas un programmēšanas apguvei. Tā vienkāršā, taču funkcionālā uzbūvē ļauj demonstrēt daudzkāju gaitas principus un izprast kinemātikas pielietojumu praksē. Robots spēj stabili pārvietoties uz līdzenas virsmas, un to iespējams vadīt attālināti ar seriālo portu vai Bluetooth moduli.



3.1. attēls Četru kāju robots

Sistēmas efektivitāti var tālāk uzlabot, aizvietojot esošos SG90 servomotorus ar SG90S modeli, kam ir metāla zobratu serdenis. Tas būtiski palielinātu mehānisko izturību un ilgmūžību, jo SG90 plastmasas zobrati mēdz nodilt intensīvas lietošanas laikā.

Lai uzlabotu sistēmas veiktspēju, viena no būtiskākajām izmaiņām būtu optimizēt programmatūras aiztures. Pašlaik kodā bieži izmantota **delay()** funkcija, kas bloķē visa robota izpildi un kavē reakciju uz lietotāja komandām. Aizvietojot šo funkciju ar **millis()** balstītu taimera loģiku vai īsākiem, nebloķējošiem cikliem, iespējams būtiski samazināt reakcijas laiku, padarot robotu piemērotāku reāllaika vadībai un ātrākai kustību secībai. Tas īpaši noderētu, ja robots tiktu paplašināts ar sensoriem vai autonomu režīmu.

Papildu iespējas ietver sensoru integrēšanu, mākslīgā intelekta algoritmus un autonomas navigācijas funkcijas, padarot šo platformu vēl pielietojamāku reālos uzdevumos un turpmākā attīstībā.

## SECINĀJUMI

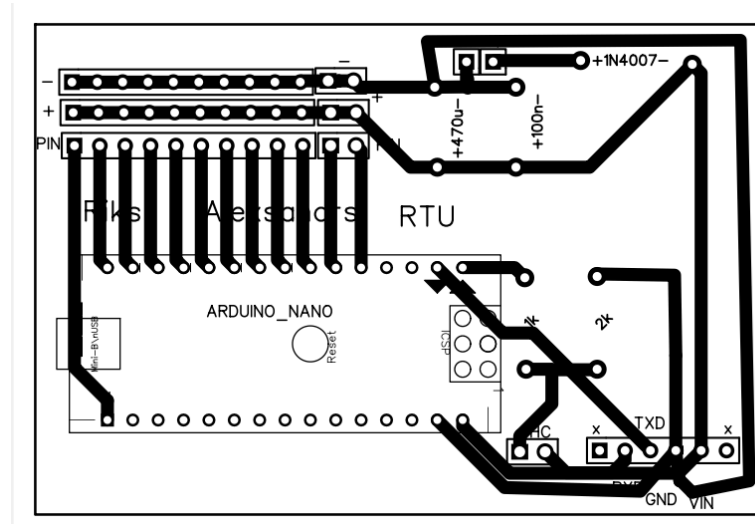
Izstrādātais četru kāju robots ir veiksmīgi īstenots gan mehāniskā, gan elektroniskā, gan programmatūras līmenī. Projektā apvienotas vairākas inženiertehniskās disciplīnas – mehānika, elektrotehnika un programmēšana –, kas ļāva iegūt vispusīgu pieredzi.

Robota konstrukcija tika veiksmīgi realizēta ar 3D drukas palīdzību, savukārt vadības sistēma izveidota, izmantojot Arduino platformu un specializētas bibliotēkas. Programmatūra tika strukturēta loģiski, ar iespējām to viegli paplašināt. Koda funkcionālie bloki nodrošina servo motoru koordinētu kustību, OLED displeja vizualizāciju un komandu apstrādi caur seriālo interfeisu. Izmantojot Bluetooth savienojumu, robots var tikt vadīts attālināti, kas padara to piemērotu arī reālās situācijās izmantošanai vai demonstrācijām.

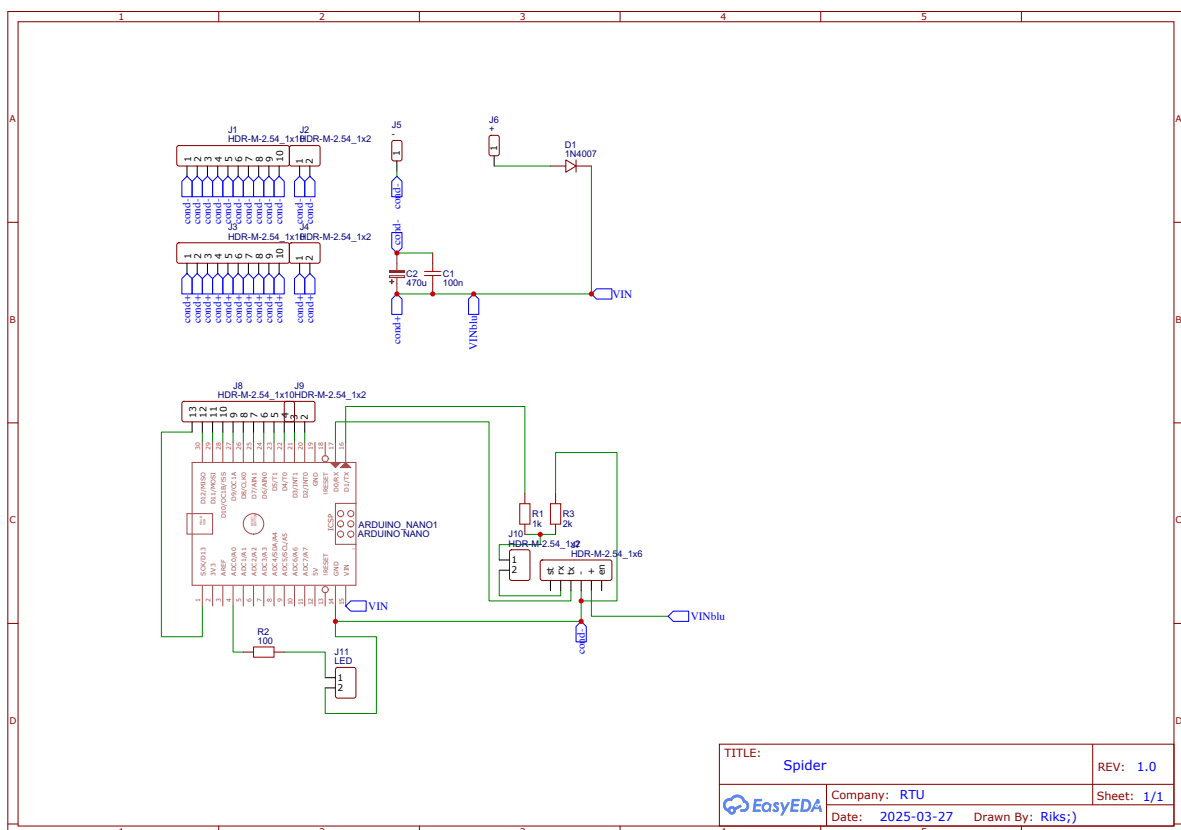
Darbs izrādījās ļoti interesants, un tā gaitā radās vairākas jaunas idejas par iespējamiem uzlabojumiem. Robots pierādīja sevi kā labs izmēģinājuma prototips lielākiem projektiem, kur nepieciešama mobilitāte un reāllaika reakcija. Nākotnē to iespējams izmantot arī kā bāzi autonomiem vai pusautonomiem robotikas risinājumiem ar mākslīgo intelektu. Gatavu rezultātu (skat. 2.pielikums).

# PIELIKUMI

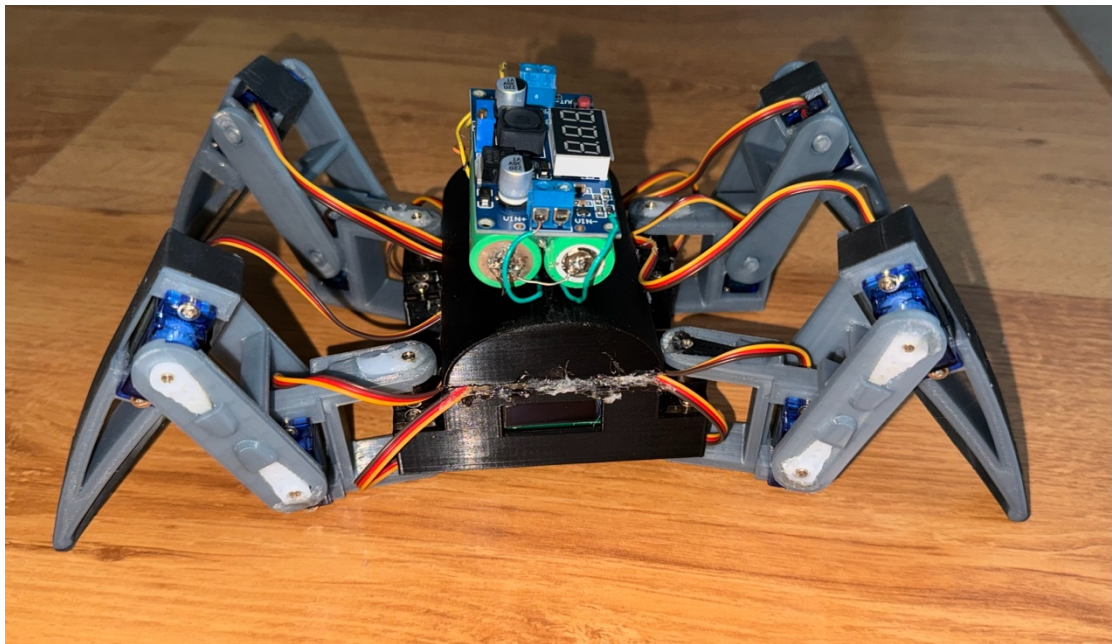
1.pielikums



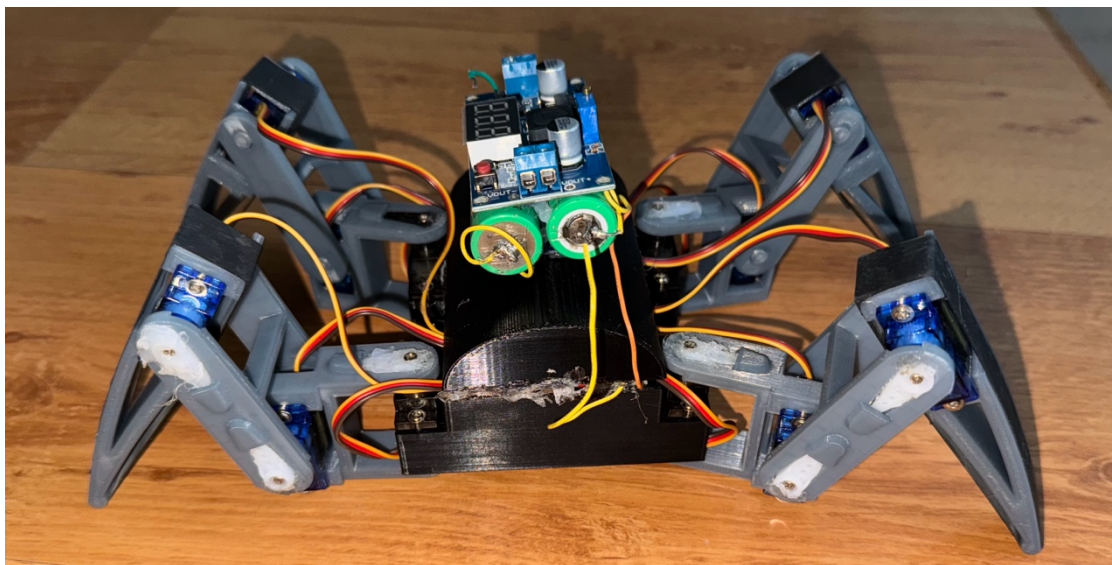
0.1.attēls EasyEDA plates savienojumi



0.2.attēls EasyEDA plates shematiskis zīmējums



0.3. attēls ČETRU KĀJU robots 1



0.4. attēls ČETRU KĀJU robots 2



Koda piemērs (Servomotori):

```
#include <Servo.h>
#include <FlexiTimer2.h>
#include <SerialCommand.h>
SerialCommand SCmd;

Servo servo[4][3];
const int servo_pin[4][3] = { {2, 3, 4}, {5, 6, 7}, {8, 9, 10}, {11, 12, 13}
};
const float length_a = 55;
const float length_b = 77.5;
const float length_c = 27.5;
const float length_side = 71;
const float z_absolute = -28;
const float z_default = -50, z_up = -30, z_boot = z_absolute;
const float x_default = 62, x_offset = 0;
const float y_start = 0, y_step = 40;
volatile float site_now[4][3];
volatile float site_expect[4][3];
float temp_speed[4][3];
float move_speed;
float speed_multiple = 1;
const float spot_turn_speed = 4;
const float leg_move_speed = 8;
const float body_move_speed = 3;
const float stand_seat_speed = 1;

void setup()
{
    Serial.begin(9600);
    Serial.println("Robot starts ");
    SCmd.addCommand("w", action_cmd);

    SCmd.setDefaultHandler(unrecognized);
    Serial.print("X now: ");
    Serial.print(site_now[0][0]);

    set_site(0, x_default - x_offset, y_start + y_step, z_boot);
    set_site(1, x_default - x_offset, y_start + y_step, z_boot);
    set_site(2, x_default + x_offset, y_start, z_boot);
    set_site(3, x_default + x_offset, y_start, z_boot);
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            site_now[i][j] = site_expect[i][j];
        }
    }
}
```

```

    FlexiTimer2::set(20, servo_service);
    FlexiTimer2::start();
    Serial.println("Servo service started");
    servo_attach();
    Serial.println("Servos initialized");
    Serial.println("Robot initialization Complete");
}

void loop()
{
    SCmd.readSerial();
}

void do_test(void)
{
    Serial.println("Piecelties");
    stand();
    delay(2000);
    Serial.println("Solis uz prieksu");
    step_forward(5);
    delay(2000);
    Serial.println("Solis atpakaļgaita");
    step_back(5);
    delay(2000);
    Serial.println("Pa kreisi");
    turn_left(5);
    delay(2000);
    Serial.println("Pa labi");
    turn_right(5);
    delay(2000);
    Serial.println("Pamaj ar roku/kaju");
    hand_wave(3);
    delay(2000);
    Serial.println("Apsesties");
    sit();
    delay(5000);
}

#define W_STAND_SIT      0
#define W_FORWARD        1
#define W_BACKWARD       2
#define W_LEFT           3
#define W_RIGHT          4
#define W_WAVE           5
void action_cmd(void)
{
    char *arg;
    int action_mode, n_step;
    Serial.println("Action:");
    arg = SCmd.next();

```

```

action_mode = atoi(arg);
arg = SCmd.next();
n_step = atoi(arg);

switch (action_mode)
{
    case W_FORWARD:
        Serial.println("Step forward");
        if (!is_stand())
            stand();
        step_forward(n_step);
        break;
    case W_BACKWARD:
        Serial.println("Step back");
        if (!is_stand())
            stand();
        step_back(n_step);
        break;
    case W_LEFT:
        Serial.println("Turn left");
        if (!is_stand())
            stand();
        turn_left(n_step);
        break;
    case W_RIGHT:
        Serial.println("Turn right");
        if (!is_stand())
            stand();
        turn_right(n_step);
        break;
    case W_STAND_SIT:
        Serial.println("1:up,0:dn");
        if (n_step)
            stand();
        else
            sit();
        break;
    case W_SHAKE:
        Serial.println("Hand shake");
        hand_shake(n_step);
        break;
    case W_WAVE:
        Serial.println("Hand wave");
        hand_wave(n_step);
        break;
    default:
        Serial.println("Error");
        break;
}
}

```

```

// w 0 1
bool is_stand(void)
{
    if (site_now[0][2] == z_default)
        return true;
    else
        return false;
}

// w 0 0
void sit(void)
{
    move_speed = stand_seat_speed;
    for (int leg = 0; leg < 4; leg++)
    {
        set_site(leg, KEEP, KEEP, z_boot);
    }
    wait_all_reach();
}

// w 3 1
void turn_left(unsigned int step)
{
    move_speed = spot_turn_speed;
    while (step-- > 0)
    {
        if (site_now[3][1] == y_start)
        {
            set_site(3, x_default + x_offset, y_start, z_up);
            wait_all_reach();

            set_site(0, turn_x1 - x_offset, turn_y1, z_default);
            set_site(1, turn_x0 - x_offset, turn_y0, z_default);
            set_site(2, turn_x1 + x_offset, turn_y1, z_default);
            set_site(3, turn_x0 + x_offset, turn_y0, z_up);
            wait_all_reach();

            set_site(3, turn_x0 + x_offset, turn_y0, z_default);
            wait_all_reach();

            set_site(0, turn_x1 + x_offset, turn_y1, z_default);
            set_site(1, turn_x0 + x_offset, turn_y0, z_default);
            set_site(2, turn_x1 - x_offset, turn_y1, z_default);
            set_site(3, turn_x0 - x_offset, turn_y0, z_default);
            wait_all_reach();

            set_site(1, turn_x0 + x_offset, turn_y0, z_up);
            wait_all_reach();

            set_site(0, x_default + x_offset, y_start, z_default);
            set_site(1, x_default + x_offset, y_start, z_up);
            set_site(2, x_default - x_offset, y_start + y_step, z_default);

```

```

        set_site(3, x_default - x_offset, y_start + y_step, z_default);
        wait_all_reach();

        set_site(1, x_default + x_offset, y_start, z_default);
        wait_all_reach();
    }
    // w 4 1
void turn_right(unsigned int step)
{
    move_speed = spot_turn_speed;
    while (step-- > 0)
    {
        if (site_now[2][1] == y_start)
        {
            set_site(2, x_default + x_offset, y_start, z_up);
            wait_all_reach();

            set_site(0, turn_x0 - x_offset, turn_y0, z_default);
            set_site(1, turn_x1 - x_offset, turn_y1, z_default);
            set_site(2, turn_x0 + x_offset, turn_y0, z_up);
            set_site(3, turn_x1 + x_offset, turn_y1, z_default);
            wait_all_reach();

            set_site(2, turn_x0 + x_offset, turn_y0, z_default);
            wait_all_reach();

            set_site(0, turn_x0 + x_offset, turn_y0, z_default);
            set_site(1, turn_x1 + x_offset, turn_y1, z_default);
            set_site(2, turn_x0 - x_offset, turn_y0, z_default);
            set_site(3, turn_x1 - x_offset, turn_y1, z_default);
            wait_all_reach();

            set_site(0, turn_x0 + x_offset, turn_y0, z_up);
            wait_all_reach();

            set_site(0, x_default + x_offset, y_start, z_up);
            set_site(1, x_default + x_offset, y_start, z_default);
            set_site(2, x_default - x_offset, y_start + y_step, z_default);
            set_site(3, x_default - x_offset, y_start + y_step, z_default);
            wait_all_reach();

            set_site(0, x_default + x_offset, y_start, z_default);
            wait_all_reach();
        }
    }
    //w 1 1
void step_forward(unsigned int step)
{
    move_speed = leg_move_speed;
    while (step-- > 0)
    {
        if (site_now[2][1] == y_start)

```

```

{
    set_site(2, x_default + x_offset, y_start, z_up);
    wait_all_reach();
    set_site(2, x_default + x_offset, y_start + 2 * y_step, z_up);
    wait_all_reach();
    set_site(2, x_default + x_offset, y_start + 2 * y_step, z_default);
    wait_all_reach();

    move_speed = body_move_speed;

    set_site(0, x_default + x_offset, y_start, z_default);
    set_site(1, x_default + x_offset, y_start + 2 * y_step, z_default);
    set_site(2, x_default - x_offset, y_start + y_step, z_default);
    set_site(3, x_default - x_offset, y_start + y_step, z_default);
    wait_all_reach();

    move_speed = leg_move_speed;

    set_site(1, x_default + x_offset, y_start + 2 * y_step, z_up);
    wait_all_reach();
    set_site(1, x_default + x_offset, y_start, z_up);
    wait_all_reach();
    set_site(1, x_default + x_offset, y_start, z_default);
    wait_all_reach();
}
//w 0 2
void step_back(unsigned int step)
{
    move_speed = leg_move_speed;
    while (step-- > 0)
    {
        if (site_now[3][1] == y_start)
        {
            set_site(3, x_default + x_offset, y_start, z_up);
            wait_all_reach();
            set_site(3, x_default + x_offset, y_start + 2 * y_step, z_up);
            wait_all_reach();
            set_site(3, x_default + x_offset, y_start + 2 * y_step, z_default);
            wait_all_reach();

            move_speed = body_move_speed;

            set_site(0, x_default + x_offset, y_start + 2 * y_step, z_default);
            set_site(1, x_default + x_offset, y_start, z_default);
            set_site(2, x_default - x_offset, y_start + y_step, z_default);
            set_site(3, x_default - x_offset, y_start + y_step, z_default);
            wait_all_reach();

            move_speed = leg_move_speed;

            set_site(0, x_default + x_offset, y_start + 2 * y_step, z_up);

```

```

        wait_all_reach();
        set_site(0, x_default + x_offset, y_start, z_up);
        wait_all_reach();
        set_site(0, x_default + x_offset, y_start, z_default);
        wait_all_reach();
    }
}

}

//w 0 5
void hand_wave(int i)
{
    float x_tmp;
    float y_tmp;
    float z_tmp;
    move_speed = 1;
    if (site_now[3][1] == y_start)
    {
        body_right(15);
        x_tmp = site_now[2][0];
        y_tmp = site_now[2][1];
        z_tmp = site_now[2][2];
        move_speed = body_move_speed;
        for (int j = 0; j < i; j++)
        {
            set_site(2, turn_x1, turn_y1, 50);
            wait_all_reach();
            set_site(2, turn_x0, turn_y0, 50);
            wait_all_reach();
        }
        set_site(2, x_tmp, y_tmp, z_tmp);
        wait_all_reach();
        move_speed = 1;
        body_left(15);
    }
}

void cartesian_to_polar(volatile float &alpha, volatile float &beta,
volatile float &gamma, volatile float x, volatile float y, volatile float z)
{
    float v, w;
    w = (x >= 0 ? 1 : -1) * (sqrt(pow(x, 2) + pow(y, 2)));
    v = w - length_c;
    alpha = atan2(z, v) + acos((pow(length_a, 2) - pow(length_b, 2) + pow(v,
2) + pow(z, 2)) / 2 / length_a / sqrt(pow(v, 2) + pow(z, 2)));
    beta = acos((pow(length_a, 2) + pow(length_b, 2) - pow(v, 2) - pow(z, 2))
/ 2 / length_a / length_b);
    //calculate x-y-z degree
    gamma = (w >= 0) ? atan2(y, x) : atan2(-y, -x);
    //trans degree pi->180
    alpha = alpha / pi * 180;
    beta = beta / pi * 180;

```

```

    gamma = gamma / pi * 180;
}

void polar_to_servo(int leg, float alpha, float beta, float gamma)
{
    if (leg == 0) {
        alpha = constrain(90 - alpha, 30, 150);
        beta = constrain(beta, 30, 150);
        gamma = constrain(gamma + 90, 30, 150);
    }

    else if (leg == 1)
    {
        alpha += 90;
        beta = 180 - beta;
        gamma = 90 - gamma;
    }
    else if (leg == 2)
    {
        alpha += 90;
        beta = 180 - beta;
        gamma = 90 - gamma;
    }
    else if (leg == 3)
    {
        alpha = 90 - alpha;
        beta = beta;
        gamma += 90;
    }

    servo[leg][0].write(alpha);
    servo[leg][1].write(beta);
    servo[leg][2].write(gamma);
}

void wait_reach(int leg) {
while (1)
    if (site_now[leg][0] == site_expect[leg][0])
        if (site_now[leg][1] == site_expect[leg][1])
            if (site_now[leg][2] == site_expect[leg][2])
                break;
}

void wait_all_reach(void) {
for (int i = 0; i < 4; i++)
wait_reach(i);
}

```



Koda piemērs (Oled displejs):

```
const unsigned long eventInterval = 1000;
unsigned long previousTime = 0;
unsigned long currentTime = millis();
#include <Servo.h>
Servo servo[4][3];

const int servo_pin[4][3] = { {2, 3, 4}, {5, 6, 7}, {8, 9, 10}, {11, 12, 13}
};

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32

#define OLED_RESET 4
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define EYE 12
#define PUPIL 5
#define TICK 1000
#define EXR 35
#define EXL 95

#define LOGO_HEIGHT 16
#define LOGO_WIDTH 16
static const unsigned char PROGMEM logo_bmp[] =
{ 0b00000000, 0b11000000,
  0b00000001, 0b11000000,
  0b00000001, 0b11000000,
  0b00000011, 0b11100000,
  0b11110011, 0b11100000,
  0b11111110, 0b11111000,
  0b01111110, 0b11111111,
  0b00111001, 0b10011111,
  0b00011111, 0b11111100,
  0b00001101, 0b01110000,
  0b00011011, 0b10100000,
  0b00111111, 0b11100000,
  0b00111111, 0b11110000,
  0b01111100, 0b11110000,
  0b01110000, 0b01110000,
  0b00000000, 0b00110000 };
```

```

void setup() {

  for (int i = 0; i < 4; i++)
  {
    for (int j = 0; j < 3; j++)
    {
      servo[i][j].attach(servo_pin[i][j]);

    }

    if (currentTime - previousTime >= eventInterval) {
      previousTime = currentTime;
    }
  }

  Serial.begin(115200);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }
  display.clearDisplay();
  display.display();

  int vpos = 18;
  display.drawRect(10, 10, 118, 20, WHITE);
  display.display();
  for(int16_t i=1; i<114; i+=2) {
    display.fillRect(12, 12, i, 16, WHITE);
    display.display();
    if (currentTime - previousTime >= eventInterval) {
      previousTime = currentTime;
    }
  }
  if (currentTime - previousTime >= eventInterval) {

    previousTime = currentTime;

  }
}

void drawEyes(int plh,int prh,int plv,int vpos){
  display.clearDisplay();
  display.fillCircle(EXR, vpos, EYE, WHITE);
  display.fillCircle(EXL, vpos, EYE, WHITE);
  display.fillCircle(EXR+plh, vpos+plv, PUPIL, BLACK);
  display.fillCircle(EXL+prh, vpos+plv, PUPIL, BLACK);
}

void displayTick(){
  display.display();
  if (currentTime - previousTime >= eventInterval) {

```

```

        previousTime = currentTime;
    }
}

void eyelid(int vpos){
    display.clearDisplay();
    display.drawCircle(EXR, vpos, EYE, WHITE);
    display.drawCircle(EXL, vpos, EYE, WHITE);
    display.fillRect(0, 0, 132, 25, BLACK);
    display.display();
    if (currentTime - previousTime >= eventInterval) {

        previousTime = currentTime;
    }
}

void centerDraw(int vpos){
    drawEyes(1,-1,0,vpos);
}

void center(int vpos){
    centerDraw(vpos);
    displayTick();
}

void left(int vpos){
    drawEyes(-3,-3,0,vpos);
    displayTick();
}

void right(int vpos){
    drawEyes(3,3,0,vpos);
    displayTick();
}

void up(int vpos){
    drawEyes(1,-1,-3,vpos);
    displayTick();
}

void down(int vpos){
    drawEyes(1,-1,3,vpos);
    displayTick();
}

void lookLeft(int vpos){
    left(vpos);
    center(vpos);
}

void lookRight(int vpos){
    right(vpos);

```

```

    center(vpos);
}

void confuse(int vpos){
    drawEyes(4,-4,0,vpos);
    displayTick();
}

void eyeblink(int vpos){
    centerDraw(vpos);
    for(int16_t i=0; i<30; i+=5) {
        display.fillRect(0, 0, 132, 10+i, BLACK);
        display.display();
        if (currentTime - previousTime >= eventInterval) {

            previousTime = currentTime;
        }
    }
}

void eyelidBlink(int vpos){
    center(vpos);
    eyelid(vpos);
    center(vpos);
}

int vpos=18;
int timeRND=0;
int actionRND=0;

void loop() {

for (int i = 0; i < 4; i++)
{
    for (int j = 0; j < 3; j++)
    {
        servo[i][j].write(90);
        if (currentTime - previousTime >= eventInterval) {
            previousTime = currentTime;
        }
    }
}

timeRND=random(5);
actionRND=random(20);
delay(TICK*timeRND);
eyelidBlink(vpos);
center(vpos);
switch (actionRND) {
case 1:

```

```

    eyeblink(vpos);
        break;
    case 2:
    eyelidBlink(vpos);
        break;
    case 3:
    confuse(vpos);
        break;
    case 4:
    up(vpos);
        break;
    case 5:
    down(vpos);
        break;
    case 6:
    left(vpos);
        break;
    case 7:
    right(vpos);
        break;
    case 8:
    lookLeft(vpos);
        break;
    case 9:
    lookRight(vpos);
        break;
    case 10:
    left(vpos);
    right(vpos);
    left(vpos);
    right(vpos);
    center(vpos);
        break;
    case 11:
    down(vpos-1);
    center(vpos);
        break;

    default:
    center(vpos);
    break;
}
}

```

## IZMANTOTA LITERATŪRA

1. Arduino. Arduino Nano [tiešsaiste]. [skatīts 2025.g. 10.apr.]. Pieejams: <https://store.arduino.cc/products/arduino-nano>
2. Adafruit SSD1306 OLED Driver Library for Arduino [tiešsaiste]. [skatīts 2025.g. 12.apr.]. Pieejams: [https://github.com/adafruit/Adafruit\\_SSD1306](https://github.com/adafruit/Adafruit_SSD1306)
3. RegisHsu Spider robot with 4 legs Arduino Nano based project [tiešsaiste]. [skatīts 2025.g. 15.apr.]. Pieejams: <http://github.com>
4. EasyEDA Online Circuit Design Tool [tiešsaiste]. [skatīts 2024.g. 20.apr.]. Pieejams: <https://easyeda.com>
5. Xiaomi. CyberDog: Xiaomi's bio-inspired quadruped robot is a bold step into the future of robotics [tiešsaiste]. [skatīts 2025.g. 30.apr.]. Pieejams: <https://www.mi.com/global/discover/article?id=2754>
6. JLCPCB. PCB Prototyping Manufacturer [tiešsaiste]. [skatīts 2025.g. 20.apr.]. Pieejams: <https://jlcpcb.com>
7. ElectronicsHub. SG90 Micro Servo Motor Basics and Control [tiešsaiste]. [skatīts 2025.g. 25.apr.]. Pieejams: <https://www.electronicshub.org/sg90-servo-motor/>
8. Random Nerd Tutorials. How to Control SG90 Servo with Arduino [tiešsaiste]. [skatīts 2025.g. 26.apr.]. Pieejams: <https://randomnerdtutorials.com/arduino-servo-motor-control/>
9. All About Circuits. HC-06 Bluetooth Module – Getting Started Guide [tiešsaiste]. [skatīts 2025.g. 27.apr.]. Pieejams: <https://www.allaboutcircuits.com/projects/control-an-arduino-via-bluetooth/>
10. Arduino Project Hub OLED Eyes Animation on SSD1306 [tiešsaiste]. [skatīts 2025.g. 28.apr.]. Pieejams: <https://create.arduino.cc/projecthub>
11. Great Projects. How to make Spider Robot using Arduino Nano | Four legged Robot with Servo Motor | Bluetooth Controlled [tiešsaiste]. [skatīts 2025.g. 30.apr.]. Pieejams: <https://www.youtube.com/watch?v=ZXjaTfm776E>